Listas em Python

Criação, acesso, métodos, funções e percurso com for

Aula 11 • 20/10/2026 • Linguagem de Programação Python (80h) • CETAM

Profa. Priscila Gonçalves

Visão geral da aula

Data

Terça-feira, 20/10/2026
Aula 11 • 4 tempos
Laboratório de informática

Habilidade técnica

Aplicar os conceitos de estrutura de dados, organização e acesso de dados, com domínio de conhecimento.

Projeto

Cadastrar vários destinos e calcular a média de gastos

Habilidade técnica

- Aplicar os conceitos de estrutura de dados com domínio
- Organizar dados em listas
- Acessar dados por índice, fatia e percurso com for
- Usar métodos e funções de listas

Objetos de conhecimento

- Estruturas de dados em Python: listas, tuplas e dicionários
- Hoje: foco nas listas
- Métodos e funções: append, remove, sort e len
- Organização e acesso de dados

Roteiro dos 4 tempos

1º tempo

Aula expositiva: listas, métodos e funções

2º tempo

Índices, fatiamento, append, remove, sort, len e for

3º tempo

Atividade: destinos, gastos, maior e menor valor

4º tempo

Desafio: vários destinos e média de gastos

1

1º tempo • Listas

Estruturas de dados, métodos e funções

Retomada da Fase 3

- Aula 08: while, contador e acumulador
- Aula 09: for e range()
- Aula 10: break, continue, pass e menu com while True
- Fase 3 concluída: laços e controle de fluxo
- Hoje: guardar vários valores com listas

Por que estruturas de dados?

- Até agora, cada variável guarda um valor
- Na agência há vários destinos e vários gastos
- Estruturas de dados organizam muitos valores juntos
- Python tem listas, tuplas e dicionários
- Hoje: listas, métodos e funções

Sem lista: variáveis soltas

```
d1 = "Manaus"
d2 = "Parintins"
d3 = "Tefé"
g1 = 120.5
g2 = 80
g3 = 200
# E se forem 50 destinos?
```

- Muitas variáveis soltas
- Difícil percorrer e somar
- A lista resolve isso

O que é uma lista?

- Coleção ordenada de valores
- Escrita entre colchetes, separada por vírgulas
- Pode guardar textos, números ou os dois
- Cada item tem uma posição (índice)
- Pode ser alterada: adicionar, remover e ordenar

Criando listas

```
destinos = ["Manaus", "Parintins", "Tefé"]
gastos = [120.5, 80, 200]
vazia = []
print(destinos)
```

- Colchetes e vírgulas
- [] cria uma lista vazia
- print mostra a lista toda

Características da lista

Ordenada

Os itens mantêm a ordem e têm índice

Mutável

Pode adicionar, remover e trocar itens

Variada

Aceita textos, números e outros tipos

Funções e métodos

- Função: recebe a lista como argumento
- Exemplos: len(lista), sum(lista), max(lista)
- Método: pertence à lista e usa ponto
- Exemplos: lista.append(x), lista.sort()
- Regra prática: método tem ponto, função não

Função e método na prática

```
nomes = ["Ana", "Bia"]
print(len(nomes))     # função
nomes.append("Caio")  # método
print(nomes)
```

- len(lista): função
- lista.append(x): método
- Método usa ponto depois da lista

Organização e acesso aos dados

- Organização: guardar valores relacionados juntos
- Acesso: por índice, por fatia ou percorrendo com for
- Busca: usar in para saber se um item existe
- Listas paralelas: destinos[i] combina com gastos[i]
- Escolher a estrutura certa facilita o programa

2

2º tempo • Explicação prática

Índices, fatiamento, append, remove, sort, len e for

Índices

```
d = ["Manaus", "Parintins", "Tefé"]
print(d[0])    # Manaus
print(d[2])    # Tefé
print(d[-1])   # Tefé
```

- O índice começa em 0
- Índice negativo conta do fim
- -1 é o último item

Cuidado com o índice

- Índices válidos: de 0 até len(lista) - 1
- Fora dessa faixa gera IndexError
- Lista com 3 itens: índices 0, 1 e 2
- d[3] causa erro; d[-1] é o último
- Confira len(lista) antes de acessar

Fatiamento

```
n = [10, 20, 30, 40, 50]
print(n[1:4])    # [20, 30, 40]
print(n[:2])     # [10, 20]
print(n[3:])     # [40, 50]
```

- [início:fim]: o fim não entra
- Sem início: do começo
- Sem fim: até o final

Alterando um item

```
gastos = [100, 200, 300]
gastos[1] = 250
print(gastos)   # [100, 250, 300]
```

- Listas são mutáveis
- Troca por índice
- Textos (str) não permitem isso

append: adicionar

```
d = ["Manaus"]
d.append("Tefé")
d.append("Coari")
print(d)
print(len(d))   # 3
```

- append adiciona no final
- Recebe um único valor
- A lista cresce a cada chamada

remove: apagar pelo valor

```
d = ["Manaus", "Tefé", "Coari"]
d.remove("Tefé")
print(d)   # Manaus e Coari
```

- remove apaga pelo valor
- Se não existir: ValueError
- pop(i) remove pelo índice

sort: ordenar

```
g = [300, 100, 200]
g.sort()
print(g)   # [100, 200, 300]
g.sort(reverse=True)
print(g)   # [300, 200, 100]
```

- sort ordena a própria lista
- reverse=True: decrescente
- sorted(g) devolve uma nova lista

len e in

```
d = ["Manaus", "Tefé"]
print(len(d))         # 2
print("Tefé" in d)    # True
print("Coari" in d)   # False
```

- len conta os itens
- in testa se o item existe
- Resultado do in é bool

Percorrendo com for

```
gastos = [120, 80, 200]
for g in gastos:
    print("Gasto:", g)
```

- for percorre cada elemento
- g recebe um valor por vez
- Sem índices nem range

Percorrendo com índice

```
d = ["Manaus", "Tefé"]
for i in range(len(d)):
    print(i, d[i])
```

- range(len(lista)) gera os índices
- Útil quando precisa da posição
- Saída: 0 Manaus e 1 Tefé

Acumulando valores da lista

```
gastos = [120, 80, 200]
total = 0
for g in gastos:
    total += g
print("Total:", total)
```

- Mesmo acumulador do while
- Resultado: 400
- Média: total / len(gastos)

Erros comuns com listas

- IndexError: índice maior que len - 1
- Esquecer vírgulas entre os itens
- Confundir lista[0] com lista[1]
- append recebe um único valor
- remove de item que não existe: ValueError

3

3º tempo • Atividade avaliativa

Destinos, gastos, maior e menor valor

Atividade avaliativa

- Três exercícios com listas
- 1. Lista de destinos
- 2. Lista de gastos
- 3. Maior e menor valor
- Entrega: arquivo .py com comentários

Exercício 1: lista de destinos

```
destinos = []
destinos.append("Parintins")
destinos.append("Novo Airão")
destinos.append("Tefé")
for d in destinos:
    print("-", d)
print("Total:", len(destinos))
```

- Começa com lista vazia
- append cadastra cada destino
- for mostra todos

Exercício 2: lista de gastos

```
gastos = [120.5, 80, 200, 45.9]
total = 0
for g in gastos:
    total += g
print("Total: R$", total)
print("Com sum:", sum(gastos))
```

- Acumulador com for
- sum soma a lista inteira
- Confira os dois resultados

Exercício 3: maior e menor

```
gastos = [120.5, 80, 200, 45.9]
print("Maior:", max(gastos))
print("Menor:", min(gastos))
```

- max e min são funções
- Maior: 200
- Menor: 45.9

Maior valor sem usar max

```
maior = gastos[0]
for g in gastos:
    if g > maior:
        maior = g
print("Maior:", maior)
```

- Lógica por trás do max
- Começa com o primeiro valor
- Atualiza ao achar um maior

Critérios da atividade

- Listas criadas e manipuladas corretamente
- Percurso com for sem erro de índice
- Maior e menor valor corretos
- Saída clara e conferida com testes
- Comentários e nomes de variáveis claros

4

4º tempo • Desafio do projeto

Vários destinos e média de gastos

Desafio: destinos e média de gastos

- Cadastrar vários destinos e o gasto de cada um
- Repetir o cadastro até o usuário digitar "sair"
- Guardar os dados em listas
- Calcular a média de gastos
- Opcional: mostrar o maior e o menor gasto

Exemplo de solução do desafio

```
destinos = []
gastos = []
while True:
    nome = input("Destino (sair): ")
    if nome == "sair":
        break
    destinos.append(nome)
    gastos.append(float(input("Gasto: R$ ")))
media = sum(gastos) / len(gastos)
print("Média: R$", media)
```

- while True e break (Aula 10)
- Duas listas paralelas
- Lista vazia dá ZeroDivisionError

Dicas para o desafio

- Teste com 3 destinos e gastos conhecidos
- Calcule a média à mão para conferir
- Trate o caso sem destinos (len(gastos) == 0)
- Mostre cada destino com seu gasto (for)
- Próximos passos: tuplas e dicionários

Fechamento

- Hoje: listas, métodos e funções
- Prática: destinos, gastos, maior e menor valor
- Desafio: vários destinos e média de gastos
- Dúvidas?
- Obrigada!